

UAVaas and DaaS: A Unified Perspective

Ryena Dhingra

References

Yapp, Justin, Remzi Seker, and Radu Babiceanu. 2016. "UAV as a Service: Enabling On-Demand Access and On-The-Fly Re-Tasking of Multi-Tenant UAVs Using Cloud Services." In *Proceedings of the 2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC 2016)*, 1–8. Sacramento, CA, USA: IEEE/AIAA.

Sellami, Mokhtar, Haithem Mezni, Hela Elmannai, and Reem Alkanhel. 2025. "Drone-as-a-Service: Proximity-Aware Composition of UAV-Based Delivery Services." *Cluster Computing* 28, no. 5 (2025): 330.



Unmanned Aerial Vehicles

- UAVs or drones
 - Military Origin
 - Reconnaissance
 - Attack missions
 - Now also used for commercial and industry operations
 - structural inspections, wildlife protection
 - emergency management, land surveying
 - real estate, film production, items delivery
 - border patrol, agriculture, flight operations management
 - FAA under pressure to safely integrate UAVs in airspace
 - Major influx expected in near future
- 

Problems

- High upfront capital cost to buy UAVs
- Licensing and certification
- High insurance premiums
- Cost of highly trained pilots
- Maintenance



Proposed Solution: UAV as a Service

- Companies do not have to own UAVs
- UAV providers install UAV hubs in areas of demand
- Populate hubs with network-aware UAVs
- Connect UAVs to a cloud service provider
- Companies temporarily connect to UAVs to perform missions
- Pay-as-you-go utility like IaaS, PaaS, SaaS



UAV as a Service (UAVaaS)

Basic Idea of UAVaaS

- Orchestration framework: decides the what, when, how
 - Tasks performed on UAVs:
 - Scheduling
 - Reservation
 - Tasking
 - Re-tasking
 - Control
 - Data consumption: how much data UAV generates and uses
 - Billing management system
 - Client can track and monitor missions
 - UAVaaS providers take care of all overhead discussed earlier
- 

A Use Case Scenario: Introduction

- Remote rural area → Cluster of villages
 - Lack of roads or transport, seasonal or unreliable roads → poor connectivity
 - Each village has a HealthCare Clinic
 - Clinics don't own drones
 - Each Clinic is connected to a UAVaaS provider through a cloud platform
 - Drones are used to cover the distance between rural clinics and laboratories, hospitals and pharmacies in nearby towns or cities
 - Used to transport medicines, samples, blood units, vaccines, etc
 - Drones may be re-tasked to deal with emergencies during disaster management
 - Cost of UAVaaS is borne by the state government
- 

Need of UAVs in Rural Healthcare

- Fly blood, urine samples rural clinics to district labs
- Deliver blood units, vaccines, medicines to rural clinics
- Monitor sanitation, flooding, drought conditions in rural areas through live video feed
- Monitor and track drone missions
- Re-task during disasters or medical emergencies
 - First-aid delivery
 - Insulin and oxygen delivery
 - Live assessment of affected villages
 - Live video feed for search and rescue operations



Categories of Users

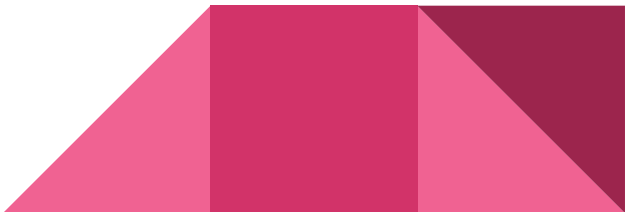
1. General operators

- Doctors, nurses, medical staff at rural clinics
- Pharmacists, hospital staff, laboratory staff in nearby town

2. Emergency service operators

- Local police, fire departments, emergency medical services (EMS)
- District Disaster Management Authority (DDMA)
- State Disaster Management Authority (SDMA)
- National Disaster Response Force (NDRF)

3. Spectators

- District health officer
 - NGO coordinators
 - Public health supervisors
 - Mass media (under certain conditions)
- 

Categories of Users

4. Ground crew personnel

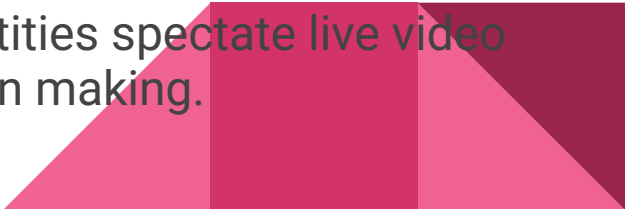
- UAVaaS provider's technicians
- Drone maintenance teams

5. Third party vendors

- Drone manufacturing companies
- Sensor manufacturers
- Payload providers (like medical box)



Users in Our Use Case Scenario

- Third party vendors manufacture and supply drones to UAV hub
 - Ground crew personnel employed by the UAVaaS provider perform intake, registration and licensing operations and put UAVs online for immediate use
 - Operators task the UAVs to deliver routine medical kits from district hospitals to rural clinics
 - Firefighters trying to extinguish a major forest fire contacts the cloud coordinator requesting a large number of UAVs
 - Since the number of requested UAVs cannot be met, the cloud coordinator selects the most appropriate UAVs to retask (with current operator's permission) for reconnaissance, planning and human life detection
 - News agencies, disaster control, and government entities spectate live video feeds, overseeing the operation and assist in decision making.
- 

Characteristics

Multi-Tenancy

- The capability of a single UAV to serve the demands of multiple customers, while still maintaining privacy of data and control
- Allows the multiple rural clinics and district health centres to use the same fleet of drones while maintaining confidentiality



Multi-Tenancy

- An operator from clinic A requests a blood sample to be delivered to the district hospital for testing
- Drone X is assigned to perform this mission
- Shortly after, clinic B makes a request for tetanus vaccines to be delivered from the district hospital
- 3 drones are assigned for this mission including drone X
- Clinic A and B can make use of the same drones without having to worry about breach of privacy



“Pay-as-you-go” billing

- Rural clinics have low budgets and do not need drones everyday
- At the same time, drones can be the perfect solution to the problem of poor healthcare in villages
- Users are charged only for the UAV resources they actually use, instead of buying drones or paying fixed monthly fees



Scheduling and Reservation

- Normal operation: pre-booking a drone in advance for some mission

Dynamic Flight Assessment

- Choosing the right drone → DONE USING DaaS



On-Demand Scalability

- 5 UAVs are active right now
- A woman develops postpartum haemorrhage
- The clinic sends a request to have blood units delivered urgently
- The cloud detects a sudden spike in demand and activates additional UAVs
- The system expands instantly without the clinic owning more drones



Policy-Based Access Control

- Different roles are enforced automatically
- Doctors and nurses at rural clinics can
 - Schedule and reserve drones
 - Request emergency deliveries
 - Can monitor mission
- Medical staff at district hospital
 - Can view monitor mission
 - Cannot control missions
- Emergency Services can
 - Have priority override
 - Control drones in case of emergencies



Policy-Based Access Control

- NGO coordinators and public health supervisors
 - Can spectate the live footage from rural areas to watch out for stagnant water bodies serving as breeding grounds for disease carrying mosquitoes, sanitation, flooding, droughts, etc
 - Can see missions taking place in real time
 - Can see rescue operations taking place
 - Cannot assign missions or control drones
- Ground crew personnel can
 - Take malfunctioning UAVs offline
 - Update firmware
 - Run diagnostics



Re-Tasking

- Upon the occurrence of an emergency, mission changes while the drone is airborne

Transparent Maintenance and Upgrade

- Users don't have to deal with repairs and updates



Safety and Security

- Data integrity
- Data confidentiality and privacy
- Session hijacking prevention
- Efficient scheduling
- Efficient re-tasking
- Smart self-preservation mechanisms
- Autonomous contingency management
- Situational awareness mechanisms
- Scalability and fault tolerance
- Send and forget memory restrictions



Abstraction at the User Level

- User can perform any task through an internet browser, desktop or mobile application that interfaces with cloud service API endpoints
 - Planning missions
 - Spectating live video feeds
 - Uploading waypoint information while reserving UAVs for a mission
- No need for
 - Specialized ground control station equipment
 - Proprietary Line of Sight (LOS) communication infrastructure



Cloud Coordinator Service

- It acts as the central coordinator or the decision making authority and decides:
 - Which UAV(s) should be used
 - When they should be used
 - Who is allowed to control or view it
 - Enforces security
 - Authenticates users and UAVs
 - Authorizes actions using policy-based access control
 - Enforces privacy between companies
 - Allows communication to take place between users and UAVs
 - Manages scheduling, reservation and re-tasking
 - Stores data such as telemetry, video, logs
- 

System Design

System Design

- 2 components:
 - UAV design
 - UAVaaS coordinator design
- UAVaaS is designed around a centralized cloud coordinator that acts as a marshaling service
- This prevents direct communication between users and UAVs and enables multi-tenancy
- Communication is broken down into 3 message types:
 - Management
 - Data
 - Control



System Design

1. Management messages

- Provide coordination and administration
- Authorization and session negotiation before flight operations and communication begin
- Termination of sessions, and disassociation of UAVs, controllers and spectators after flight operations

2. Control messages

- Light-weight
- Provide operators an interface to send control signals to the UAV
- Because UAVs fly autonomously, controllers send only high-level commands such as altitude, speed, waypoints, and camera control

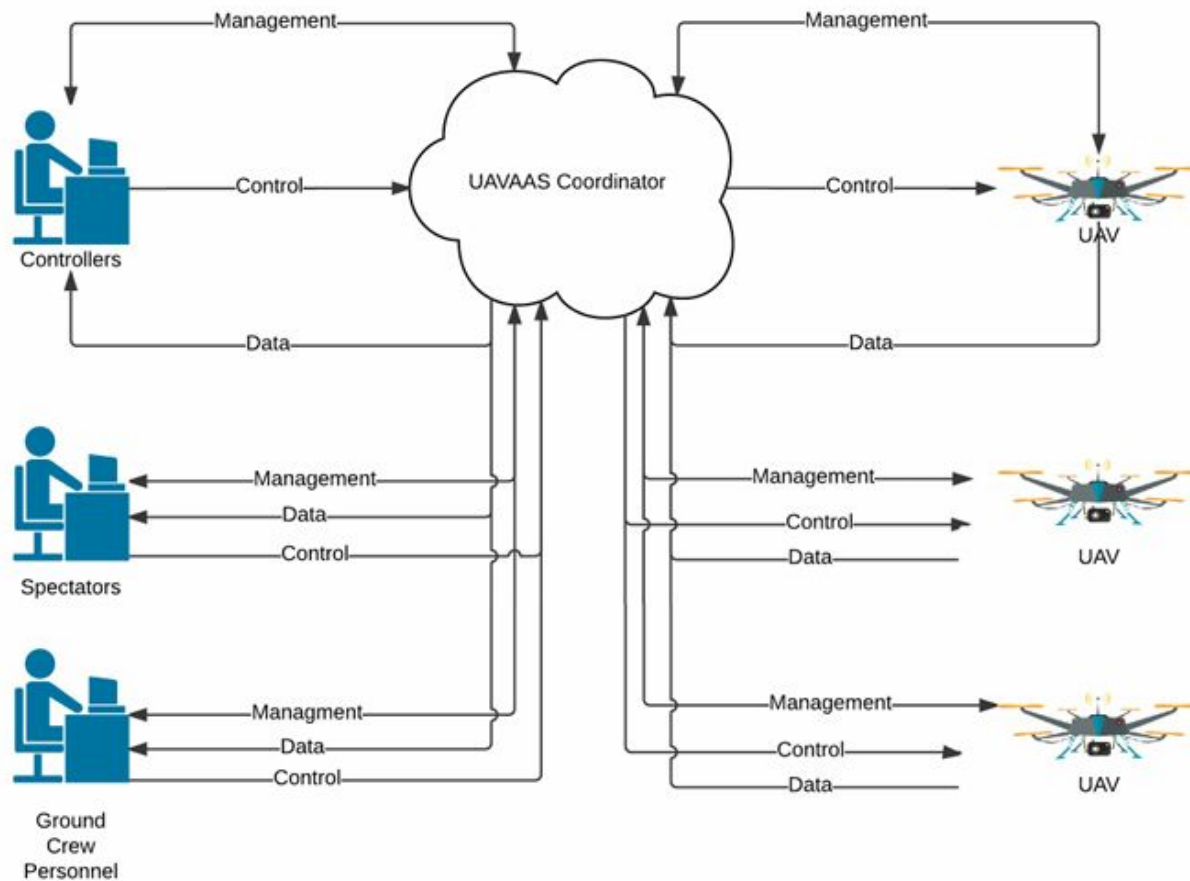


System Design

3. Data messages

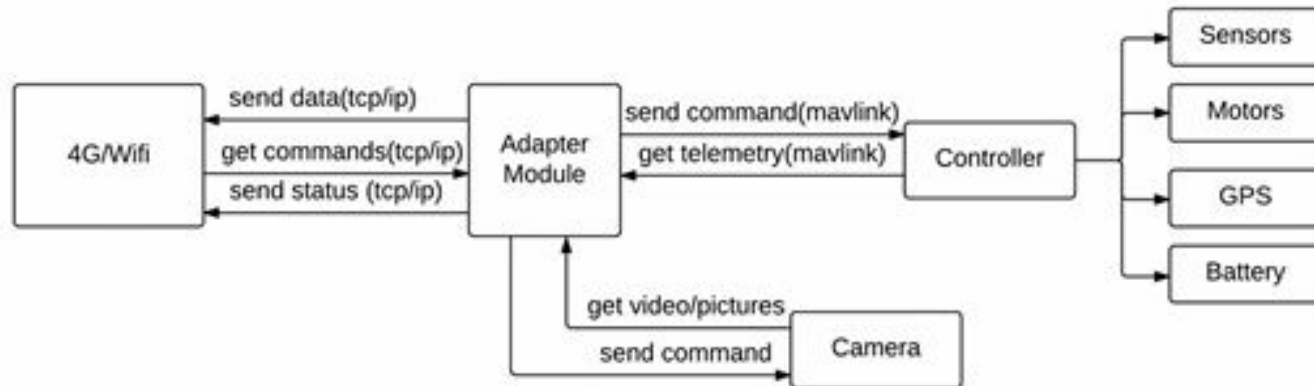
- Contain telemetry, video, pictures, warning logs or system messages
- These are sent back to the UAVaaS Coordinator which is then distributed to appropriate operators and spectators where applicable
- May include high quality photos or videos that have been taken, or real time altitude, speed and location information needed by the operator to make decisions about a current mission



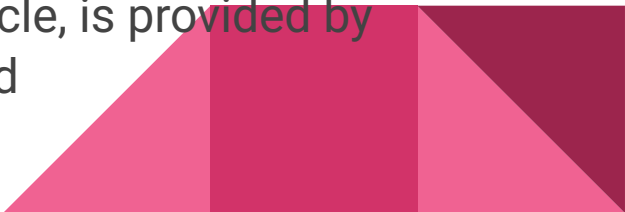


UAV Design

- The primary flight maneuvers such as takeoffs, landings, way point navigation, speed and altitude changes are handled by the UAV's on-board flight controller
 - These commands are received over 4G LTE
 - A dedicated companion board, for example an Arduino or Raspberry Pi acts as an adapter
 - The adaptor parses network packets into a protocol that the on-board flight controller understands
 - Multiple adapters can be designed to interface with different controller protocols
- 



UAV Design

- The key component of any UAV that connect to a service provider is an Adapter Module installed on a companion board
 - The UAV Adapter Module is responsible for
 - Contacting and updating the UAVaaS with status and availability information
 - Respond to new flight assignments
 - Receive and interpret control commands sent by operators
 - Package and send data messages to the UAVaaS Coordinator for distribution
 - Store public certificate and private key information needed for session negotiation
 - Authenticate to a UAVaaS Cloud Provider
 - Fail-safes such as preventing operators from disobeying speed and altitude restrictions, or intentionally trying to destroy the vehicle, is provided by additional software installed on the companion board
- 

UAVaaS Coordinator Design

- Cloud coordinator service
- Two services:
 - Management service → management messages
 - Messaging service → data and control messages
- Why two services?
 - Management messages are sent and received as a request response pair
 - Control and data messages are forwarded to the appropriate receiver



UAVaaS Coordinator Design

- **Management Service:**
 - Handles coordination and scheduling
 - sets up messaging sessions between operators, spectators and UAVs
 - Ensures only users scheduled for a given flight block may use these sessions
 - Manages user accounts via cloud databases
 - Stores mission data and flight logs securely
 - Configures messaging service based on mission needs
 - Notifies ground crew of warnings, errors, and battery status
- **Messaging service:**
 - Handles duplication and forwarding of control and data messages
 - Gives error and warning log messages to the management service
 - Uses a publisher-subscriber architecture
 - Exchanges and queues work together to send and receive messages



UAVaaS Coordinator Design

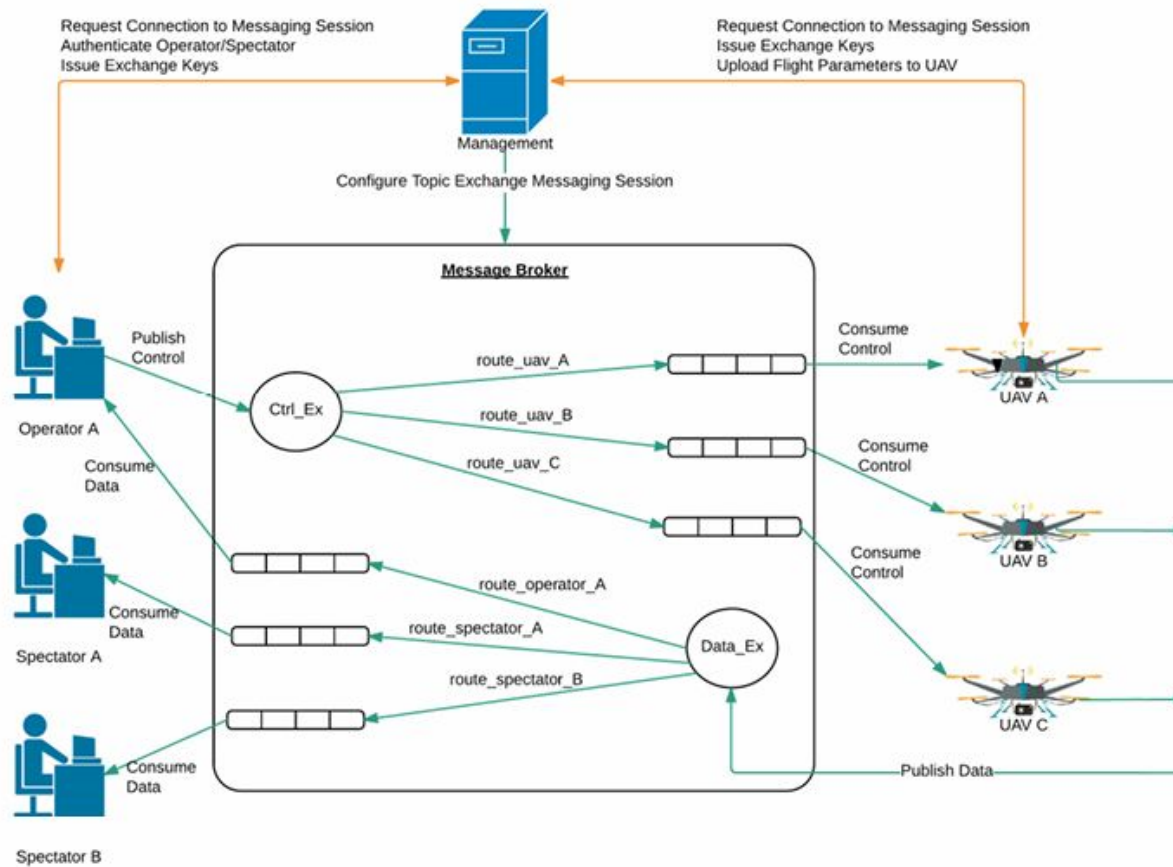
- Exchange:
 - A cloud-side routing point that decides where messages should go
 - Receives messages from publishers, looks at the message topic, forwards copies to the correct queues
 - It does not store messages
 - Ctrl Ex and Data Ex
- Queue:
 - A temporary message inbox for a specific receiver
 - Stores messages until they are read, belongs to one subscriber, ensures messages are not lost
- Topic:
 - A label to decide which queues should receive a message



UAVaaS Coordinator Design

- How does the publisher-subscriber architecture work for messaging service?
- UAVs and users do not directly talk to each other
- The cloud handles message routing and duplication
- Publishers:
 - Operators (control messages)
 - UAVs (data messages)
- Subscribers:
 - Operators (data messages)
 - Spectators (data messages)
 - UAVs (control messages)





UAVaaS Coordinator Design

- Operator requests mission and management service approves it
 - Management service creates messaging session
 - Control and data exchanges are instantiated
 - Mission-specific topics and queue IDs are generated
 - UAV, operators, and spectators create and subscribe queues to desired exchange and specify the assigned topic
 - Operator publishes control commands to control exchange
 - UAV publishes telemetry and data to data exchange
 - The exchange receives a message, checks its topic, duplicates it, and forwards it to all queues subscribed to that topic
 - Queues buffer messages for each subscriber
- 

2 Questions

- The problem we have been addressing till now:

“How can companies use drones without owning them, while ensuring security, scalability, and multi-tenancy?”

- The question that remains unanswered:

“Given many drone services distributed across space, how do we compose the best set of drones and routes to complete a delivery efficiently?”

- We answer the second question using DaaS



UAVaaS and DaaS

- UAVaaS answers:
 - How can drones be offered as a shared cloud service?
 - How do we securely connect users and drones?
 - How do multiple users share the same UAV fleet safely?
 - How are roles separated?
 - How is scalability handled?
 - How is safety enforced at a system level?
 - How is safety enforced at a system level?
 - How do drones remain usable without user intervention?
- DaaS answers:
 - Which drones should be selected for a delivery?
 - How do we handle deliveries beyond a single drone's range?
 - How can multiple drone services be composed into one mission?
 - How do we minimize delivery time, cost, or energy?
 - How do we select an efficient route?
 - How do we minimize delivery time, cost, or energy?



UAVaaS and DaaS

- UAVaaS answers system-level questions about sharing drones
- DaaS answers decision-level questions about using drones optimally



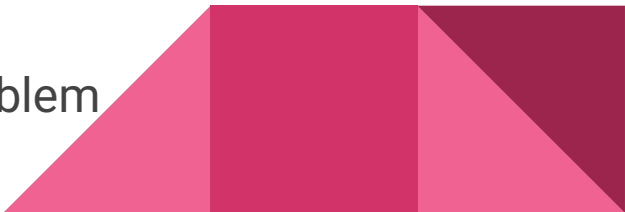
Drone as a Service (DaaS)

Introduction to DaaS

- Users request a delivery service and the system decides which drone(s) perform it
- Functional properties:
 - Set of skyway segments
- Non functional properties:
 - Battery capacity
 - Payload
 - Flight range
 - Speed
- Nodes:
 - Recharging stations
 - Pickup or delivery locations



Introduction to DaaS

- Edges:
 - Skyway segments
 - Intrinsic factors:
 - Drones's limited capacity (payload, battery, etc)
 - Extrinsic factors:
 - Weather conditions
 - Stations availability
 - To cope with these intrinsic and extrinsic factors and to achieve their missions, drones must cooperate and recharge their battery multiple times at different charging stations
 - DaaS composition is an effective solution to this problem
- 

Introduction to DaaS

- Principles of service computing
 - Drones scheduling and routing as a traditional service composition problem
 - Most current approaches are less adaptive and responsive to the dynamic UAV network
 - Use simple graphs that miss rich relationships and semantics. This ignores hidden information
 - Scalability is poor because optimization methods (MILP, evolutionary algorithms) are computationally expensive
 - Existing work optimizes either user-side constraints or provider-side constraints, not both
 - Route planning and drone assignment are treated as separate problems
 - Drone failures and station congestion are rarely modeled explicitly
- 

Knowledge graph for DaaS

- Knowledge graphs model drones, stations, and skyways with rich semantic relationships
- They capture both functional and non-functional properties ignored by simple graphs
- Embeddings map these entities into a shared vector space
- This enables proximity-aware selection of compatible drones and stations
- Hence DaaS composition and routing are optimized efficiently
- The authors model the drones' network as a knowledge graph, called DaaSKG



Knowledge graph for DaaS

- Multi-relational, hierarchical and semantic capabilities
 - Represent sky network entities and also multiple, typed, and hierarchical relationships between them
 - Evolving and inter-operable nature allows integrating new entities and relationships
 - Querying support using languages like SPARQL
 - Provides inference and reasoning capabilities
 - Network representation learning techniques like metapath2vec help map the knowledge graph entities in a vector representation, facilitating their processing based on their closeness in the embedding space
- 

Knowledge graph for DaaS

- DaaSKG:
 - Data management (e.g., drones/stations clustering)
 - Analytics queries (e.g., stations demands, failure prediction)
 - Knowledge discovery (e.g., latent connections, flight patterns, drone formation patterns)
 - Decision making (e.g., en-route update of flight paths)
 - Recommendation (e.g., best drone services with low failure rates)



Problem Formulation

Given a set of DaaS with their capacities and features, a set of stations with their charging features, a set of delivery tasks with their cost and time constraints, the goal is to find the optimal skyway path and set of drone services that deliver the consumer's requested packages in a minimum time and cost, while taking into account the drones capacities and the delivery constraints



Problem Formulation

- Input:

- Set of drones, $D = \{d_1, d_2, \dots, d_n\}$
- Set of delivery services, $S = \{daas_1, daas_2, \dots, daas_n\}$
- Set of recharging stations, $RS = \{rs_1, rs_2, \dots, rs_n\}$
- Each rs_i hosts a set of recharging pads, $P = \{p_1, p_2, \dots, p_n\}$

- Output:

- Composite DaaS, $CDaaS = \{daas_1, daas_2, \dots, daas_n\}$
- A set of skyway segments representing the delivery path from a source node to a destination



Definitions

- Delivery request, $DR = \langle DS, DT, D^F, D^q \rangle$
 - $DS \rightarrow$ delivery source
 - $DT \rightarrow$ delivery target
 - $D^F = \{it^1, it^2, \dots, it^n\} \rightarrow$ set of items in the package
 - Each item is characterised by weight and quantity
 - $D^q = \langle T, cost \rangle \rightarrow$ consumer's time and cost constraint
 - $T = [\tau_b, \tau_e]$



Definitions

- Each drone-as-a-service, $daas = \langle F^s, Q^s \rangle$
 - $F^s = \{f^s_1, f^s_2, \dots, f^s_n\} \rightarrow$ set of delivery operations offered by a drone (e.g., first-aid kits)
 - $Q^s = \{q^s_1, q^s_2, \dots, q^s_n\} \rightarrow$ set of non-functional properties of the delivery service DaaS (e.g., delivery cost, payload, max speed, flying range, battery capacity)
- Recharging station, $rs = \langle F^r, P \rangle$
 - $F^r = \{f^r_1, f^r_2, \dots, f^r_n\} \rightarrow$ set of recharging station's features including the max power, the capacity and the type of station (mobile or fixed)
 - Set of recharging pads, $P = \{p_1, p_2, \dots, p_n\}$

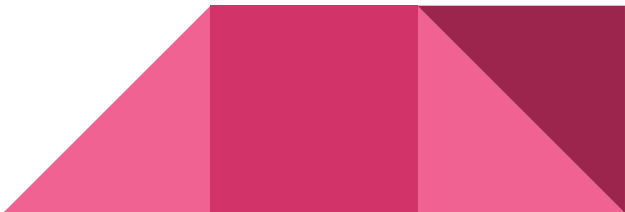
Definitions

$$Delivery : D^F \rightarrow DaaS \quad (1)$$

where $it_j \mapsto Delivery(it_j) = daas_i, 1 \leq i \leq n, 1 \leq j \leq w$.

$$Path : DaaS \rightarrow Segment \quad (2)$$

where $daas_i \mapsto Path(daas_i) = Seg_k, 1 \leq i \leq n, 1 \leq k \leq p$.



Definitions

- Skyway segment, $\text{Seg}_{ij} = \langle rs_i, F, rs_j \rangle \rightarrow$ the connection between two stations
 - rs_i and $rs_j \in RS$
 - $F \rightarrow$ skyway segment's features, such as the distance and the wind speed
- Constraints
 - User constraints
 - Delivery cost
 - Delivery time
 - Provider constraints
 - Recharging time
 - Payload weight



Definitions

- DaaS knowledge graph, DaaS_{KG}, $G = (E, R, D^+)$
- $E = \langle E_d, E_s, E_r, E_p, E_f \rangle$
 - $E_d \rightarrow$ drone
 - $E_s \rightarrow$ delivery services
 - $E_r \rightarrow$ recharging stations
 - $E_p \rightarrow$ pads
 - $E_f \rightarrow$ features
- $R \subseteq E \times E$
- $D^+ \rightarrow$ set of triples (facts) expressing the entities interconnections

Definitions

- Fact, $f = (e_i, r, e_j)$
 - $e_i, e_j \in E$ are drones, recharging stations, delivery services or features
 - $r \in R$ is a typed relationship

- Relation

- $$f(e_i, r, e_j) = \begin{cases} \textit{ATTEND} & \text{if } e_i \in \mathcal{E}_d \wedge e_j \in \mathcal{E}_r \\ \textit{PROVIDE} & \text{if } e_i \in \mathcal{E}_d \wedge e_j \in \mathcal{E}_s \\ \textit{SkySegment} & \text{if } e_i \in \mathcal{E}_r \wedge e_j \in \mathcal{E}_r \\ \textit{D - SIMILAR} & \text{if } e_i, e_j \in \mathcal{E}_d \\ \textit{S - SIMILAR} & \text{if } e_i, e_j \in \mathcal{E}_s \\ \textit{R - SIMILAR} & \text{if } e_i, e_j \in \mathcal{E}_r \\ \textit{F - SIMILAR} & \text{if } e_i, e_j \in \mathcal{E}_f \\ \textit{BELONG} & \text{if } e_i \in \mathcal{E}_p \wedge e_i \in \mathcal{E}_r \end{cases}$$

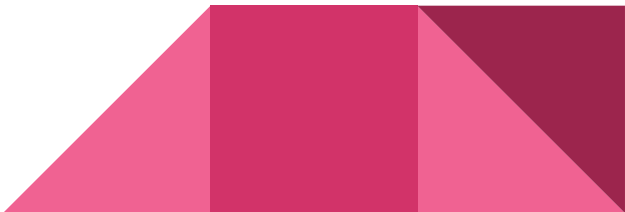
1. DaaSKG Construction

DaaSKG Construction

- To populate the DaaSKG, we mine several data sources that contain relevant information about drones (e.g., features, capacities, and delivery operations), recharging stations (e.g., locations, states, pads), and skyway segments (e.g., distance, flight restrictions)
- Such information is categorized into entities, typed relations, and auxiliary information
- Such info is combined to form the facts (triples) in the DaaSKG
- This is knowledge graph construction



DaaSKG Construction

- Knowledge graph construction
 - Knowledge population
 - Knowledge completion
 - Steps:
 - Data collection: Drone, station and skyway data are gathered from multiple sources and preprocessed through cleaning and normalization
 - Entity–relation extraction: Core entities and typed relations are identified from data patterns
 - Entity enrichment: Entities and relations are augmented with operational, QoS, and environmental attributes
 - Triple formation and inference: Knowledge is stored as triples and enriched by inferring latent relations using KGE
- 

2. DaaS Composition Process

DaaS Composition Process

1. Extraction of candidate drones and stations
2. Proximity-aware DaaS Composition
3. Skyway path selection



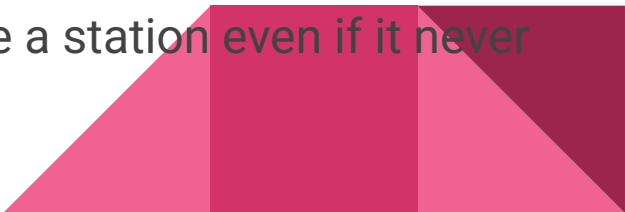
Extraction of Candidate Drones and Stations

- DaaS selection is modeled as a proximity-aware matching problem over the DaaSKG
 - Proximity helps identify suitable drones and nearby recharging stations for delivery missions
 - Knowledge graph embeddings (KGE) map DaaSKG entities into a low-dimensional vector space
 - Embeddings capture structural, contextual, and latent relationships beyond raw features
 - High-proximity drones and stations appear close in the embedding space and form a candidate subnetwork
- 

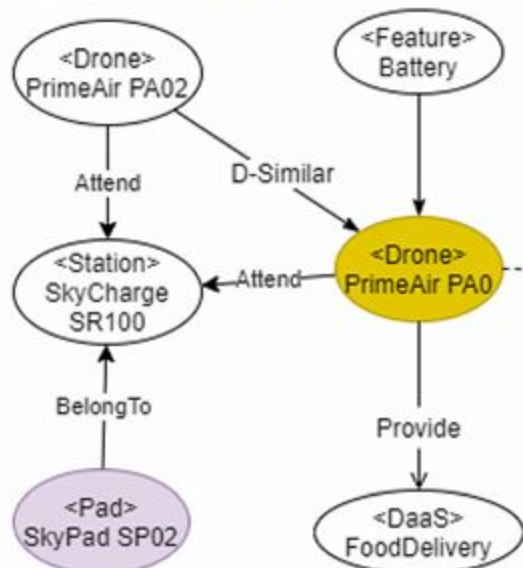
Extraction of Candidate Drones and Stations

- Low-proximity entities are excluded as they do not meet delivery or charging constraints
 - The metapath2vec model is used to compute proximity degrees via guided random walks
 - Meta-paths encode meaningful patterns such as Drone–Station–Pad and Station–Station
 - Entities with similar contexts co-occur frequently in random walks and receive similar embeddings
 - High proximity enables inference of latent relations for DaaS composition
- 

Extraction of Candidate Drones and Stations

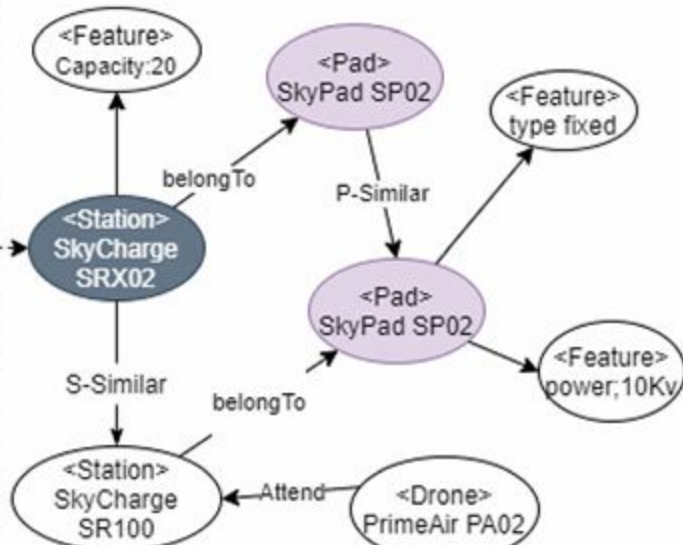
- Simple summary:
 - It converts the complex DaaS knowledge graph into numerical vectors using embeddings
 - Drones and stations with similar capabilities and requirements end up close together in this vector space
 - This closeness (proximity) tells the system which drones can work well with which stations
 - Drones or stations that do not meet delivery or charging constraints are automatically filtered out
 - Hidden but useful relationships (e.g., a drone can use a station even if it never did before) are inferred
- 

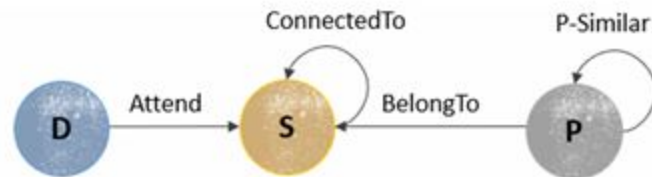
Context of entity *PrimeAir Drone*



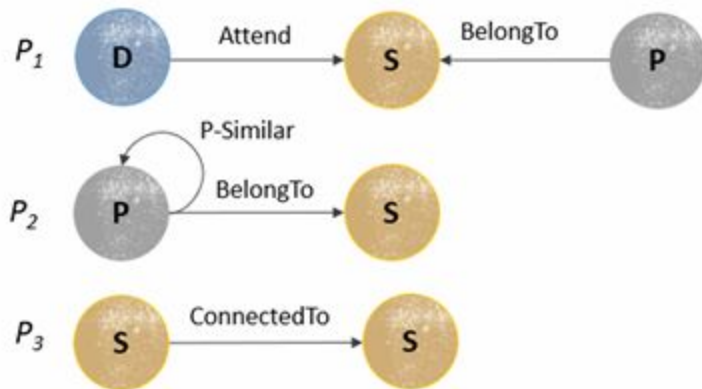
ATTEND

Context of entity *SkyCharge SRX02*

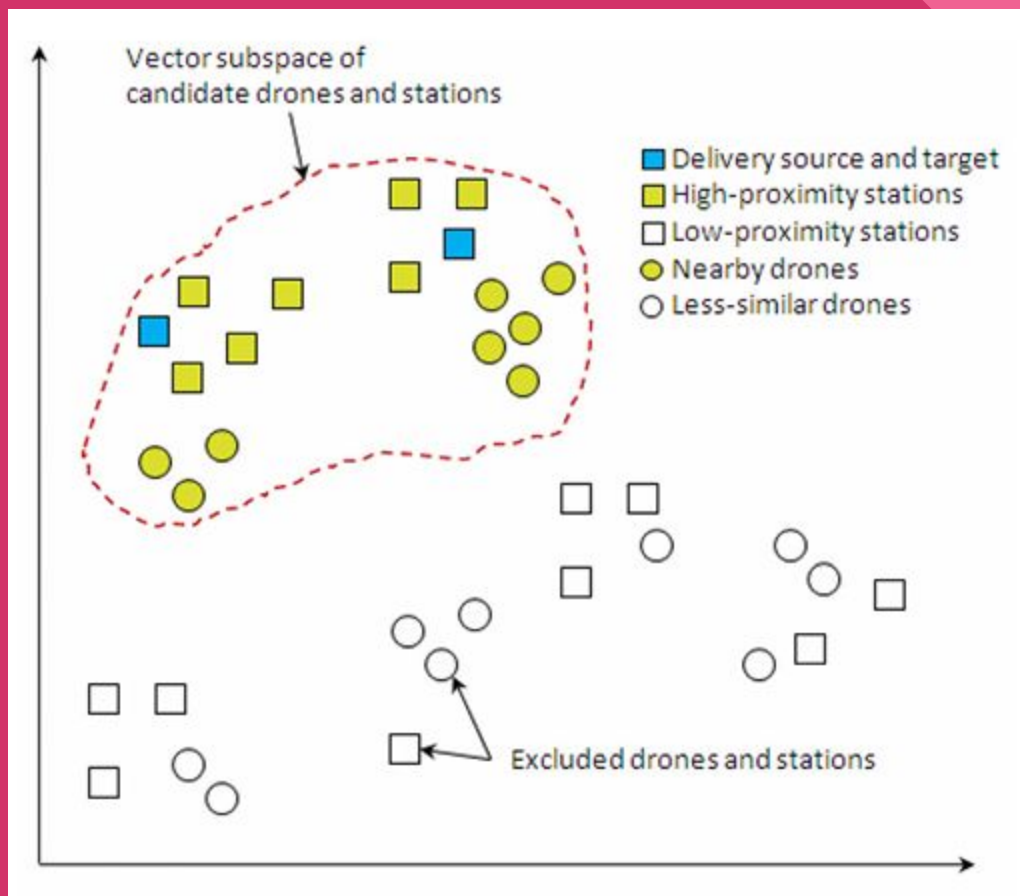


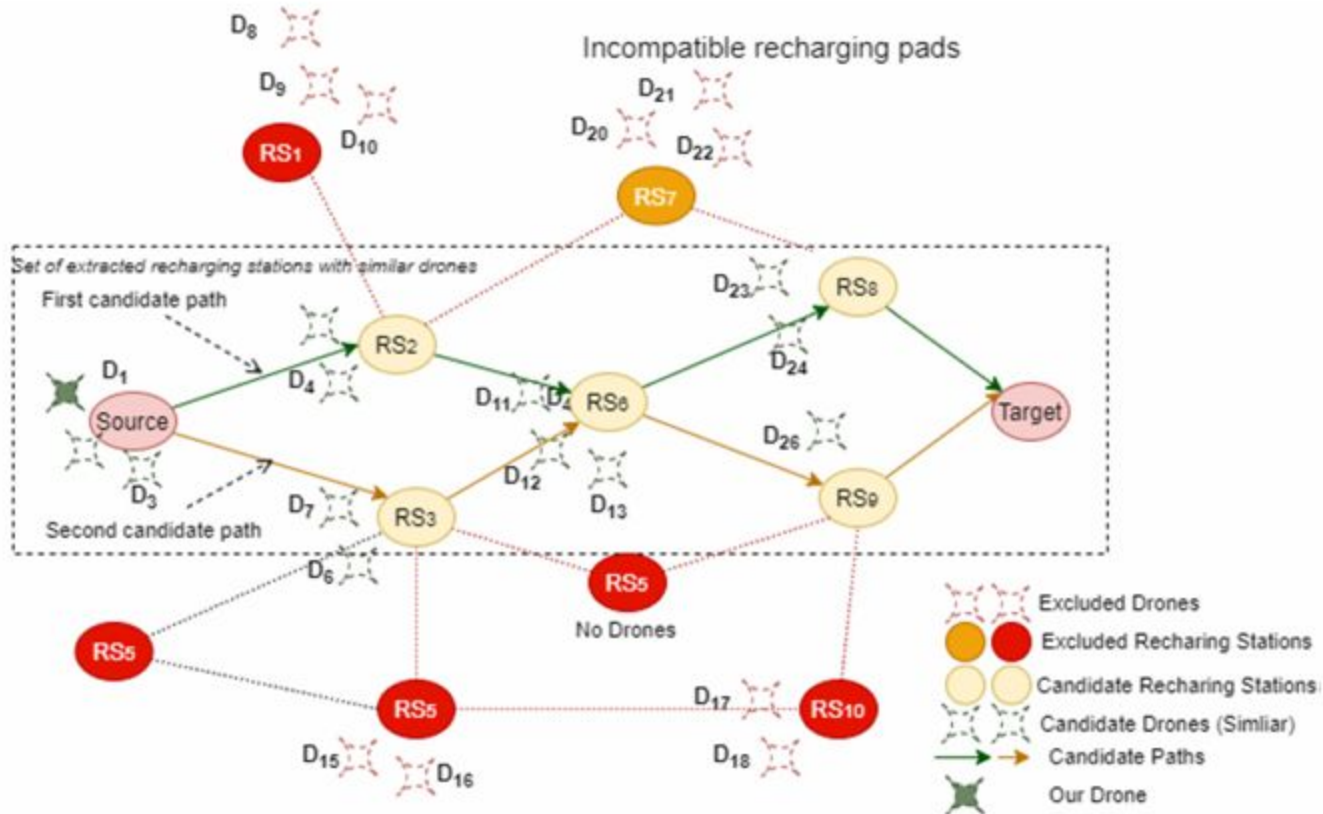


(a) Example of DaaSKG schema



(b) Examples of meta-paths





Proximity-aware DaaS Composition

- This step takes as input the previously generated embeddings of DaaS entities, as well as their proximity degrees
- The quality of a DaaS composition (a delivery mission) depends on service features. Some of these latter must be minimized (e.g., cost), while others must be maximized (e.g., flight range, battery capacity, max speed)
- A proximity-aware algorithm that takes as input a delivery request and the subset of drone services that belong to the vector subspace
- The output of Algorithm 1 is a composite DaaS, that will be exploited to determine the delivery path



Algorithm 1 Composition of drone services for delivery

Require: $\mathcal{D}_{prox}$: candidate DaaS, $\mathcal{DR}$: delivery request**Ensure:** $CDaaS_{best}$: composite DaaS

- 1: $\mathcal{D}_{prox} \leftarrow$ Sort drones using Eq. (12)
 - 2: $CDaaS \leftarrow \emptyset$ $\triangleright$ Set of candidate DaaS compositions
 - 3: $D_{temp}^F \leftarrow D^F$
 - 4: $CD \leftarrow \emptyset$
 - 5: **while** ($D_{temp}^F \neq \emptyset$ AND $Payload(\mathcal{D}_{prox}) \geq Weight(D_{temp}^F)$) **do** $\triangleright \triangleright$ Check if the remaining drones in $\mathcal{D}_{prox}$ can hold the consumer's requested package.
 - 6: **if** d_i can hold it_j **then**
 - 7: Put it_j in d_i $\triangleright \triangleright$ Place current item in drone d_i .
 - 8: Remove item it_j from D_{temp}^F .
 - 9: **else**
 - 10: Add d_i to CD $\triangleright \triangleright$ Add full drone to the CDaaS.
 - 11: Remove d_i from $\mathcal{D}_{prox}$ $\triangleright \triangleright$ Update the set of available drones.
 - 12: $i \leftarrow i + 1$
 - 13: **end if**
 - 14: **end while**
 - 15: **if** ($D_{temp}^F = \emptyset$) **then**
 - 16: add CD to $CDaaS$
 - 17: Compute score of CD using Eq. (11)
 - 18: GoTo 5 $\triangleright \triangleright$ Generate the next composite DaaS.
 - 19: **end if**
 - 20: Sort DaaS compositions in $CDaaS$
 - 21: **Return** $CDaaS_{best}$
-

Skyway Path Selection

- Algorithm 2 takes the composed CDaaS and the high-proximity recharging stations (RS_{rox}) to compute a feasible delivery path
- The function searches for valid skyway paths between the source and destination using the selected high-proximity stations



Algorithm 2 Skyway path selection

Require: $CDaaS_{best}$, $\mathcal{RS}_{prox}$, Delivery request DR .**Ensure:** Delivery path $path_{opt}$ for $CDaaS$.

```
1: Decreasing sort of  $\mathcal{RS}_{prox}$ 
2:  $Seg^{SK} \leftarrow delivery\_paths(\mathcal{RS}_{prox}, DS, DT)$ 
3: Sort  $Seg^{SK}$  in increasing order of their distance
4:  $D_a \leftarrow \emptyset$ 
5:  $S_{max} = 0$ 
6: for each  $path \in Seg^{SK}$  do
7:   Initialize  $C, T, t_f, t_r$ 
8:   for each  $seg(rs_i, rs_j) \in path$  do
9:     for each drone  $d \in rs_i$  do
10:      if  $(d.range > seg.distance)$  then
11:        if  $(D^F - d.payload) \neq \emptyset$  then
12:           $C \leftarrow C + cost(d, rs_j)$ 
13:           $t_f \leftarrow t_f + time(d, rs_j)$ 
14:           $t_r \leftarrow rechargeTime(d, rs_j)$ 
15:           $T \leftarrow T + t_f + t_r$ 
16:          add  $d$  to  $D_a$ 
17:          Break when  $(D^F = \emptyset)$ 
18:        end if
19:      end if
20:    end for
21:  end for
22:   $Score \leftarrow 1/(w_1T + w_2C + w_3|D_a|)$ 
23:  if  $(Score > S_{max})$  then
24:     $S_{max} \leftarrow Score$ 
25:     $path_{opt} \leftarrow path$ 
26:  end if
27: end for
28: Return  $path_{opt}$ 
```

UAVaaS and DaaS: Conclusion

- Layered architecture
 1. Physical UAV Layer
 - Drones, sensors, batteries and onboard controllers
 - Performs actual flying, sensing, and package carrying
 2. Communication Layer
 - Between drones and users via the cloud
 3. UAVaaS Infrastructure Layer
 - Abstracts physical drones
 - Manages multi-tenancy, access control, on-the-fly re-tasking, etc



UAVaaS and DaaS: Conclusion

4. DaaS Intelligence Layer

- Selects drones, routes, recharging stations
- Satisfies user and provider constraints

5. Knowledge Layer

- Maintains knowledge graphs, embeddings, and historical data
- Proximity-aware reasoning and decision-making

6. Application Layer

- APIs and dashboards
- Allows users to request, track, and manage delivery services

